김태호 / 개인 프로젝트

2026.02 - 2026.04

GitHub · Demo

Fundus Vision

안저 이미지로 경험한 Vision AI 개발 과정

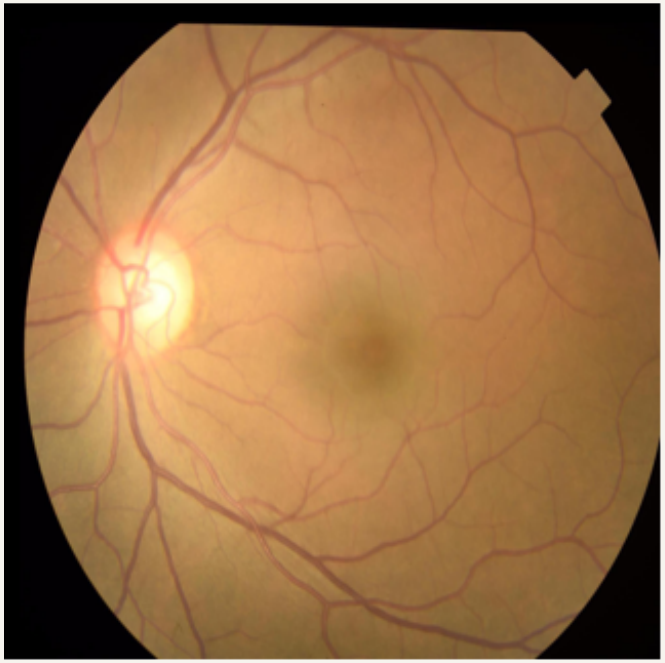
이미지 점검과 전처리,
불균형 대응 전략 비교,
오류 분석과 결과 해석,
데모 구현을 수행했습니다.

Python / PyTorch / timm
ConvNeXtV2 / Grad-CAM / Gradio

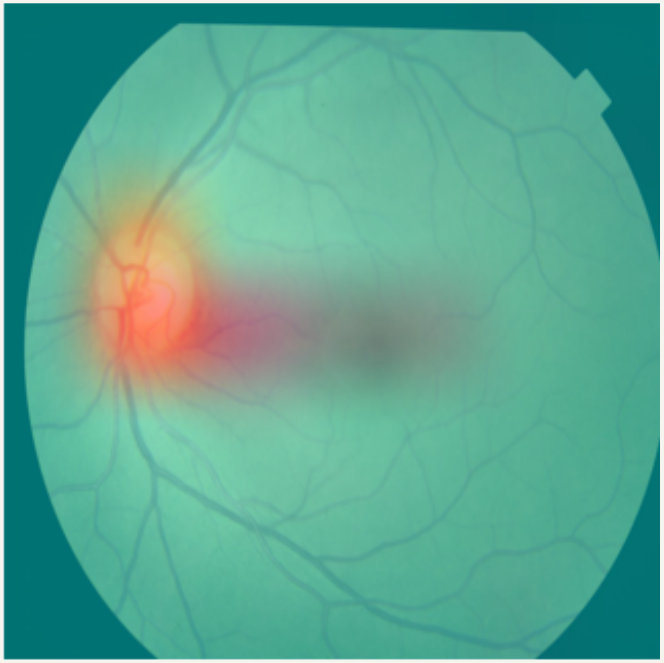
분류 결과와 주목 영역을 함께 확인

Glaucoma 저장 예시 00 / 동일 이미지의 입력과 Grad-CAM overlay

Original



Grad-CAM overlay



내부 검증 기록 / Strategy A / 3,249장

86.98%
Accuracy

0.8992
Macro F1

저장된 평가 CSV 기준. 원본·증강 분할 독립성 및 외부 검증은 미확인입니다.
연구용 프로토타입이며, Grad-CAM은 정성적 모델 점검에 사용했습니다.

데이터 전처리와 불균형 대응

클래스별 표본 차이와 이미지 품질 편차를 확인하고, 같은 기반 모델에서 두 전략을 비교했습니다.

01
데이터 점검
클래스 분포와 밝기·해상도 등
입력 이미지의 편차를 확인

02
입력 정리
RGB 변환, 크기 조정, 정규화
학습·추론용 전처리 구성

03
학습 전략
Class Weight와 Oversampling
각각에 Focal Loss 결합

04
평가와 해석
평균 지표와 클래스별 오류,
Grad-CAM을 함께 확인

데이터 소개

21,577 장 / 공개 자료에 보고된 전체 규모



원본 5,335장 증강 16,242장

Healthy 포함 10개 클래스.
내부 평가 리포트의 표본은 3,249장으로,
데이터 소개 수량과 구분해 제시했습니다.

Strategy A | Class Weight + Focal Loss

빈도가 적은 클래스의 손실 기여도를 조정하는 방향으로 학습 전략을 구성했습니다.

Strategy B | Oversampling + Focal Loss

WeightedRandomSampler로 소수 클래스의 학습 노출 빈도를 높이는 전략과 비교했습니다.

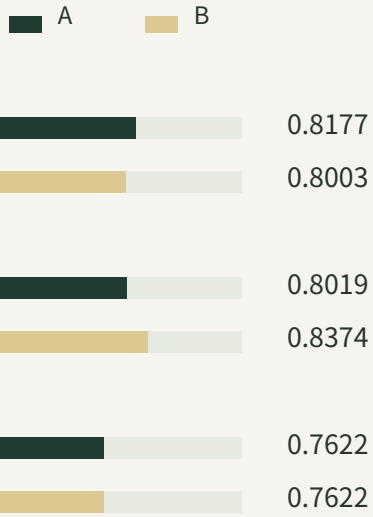
입력 크기: 공개 A/B 학습 코드 256×256, 데모 코드 224×224.
과거 평가 실행의 정확한 전처리 조건은 별도 확인이 필요합니다.

내부 검증 결과

두 결과 CSV에서 클래스별 평가 표본 수(support)의 합계는 각각 3,249장입니다. 아래는 저장된 결과의 비교입니다.

전략	Accuracy	Macro Precision	Macro Recall	Macro F1
A / Class Weight	0.8698	0.8918	0.9116	0.8992
B / Oversampling	0.8670	0.8918	0.8926	0.8914

취약 클래스의 Recall / 0-1 범위



평균 성능과 클래스별 차이

A의 Macro F1은 B보다 0.0078 높았습니다. 하지만 Healthy의 Recall은 B가 더 높아, 모든 클래스에서 한 전략이 우세하지는 않았습니다.

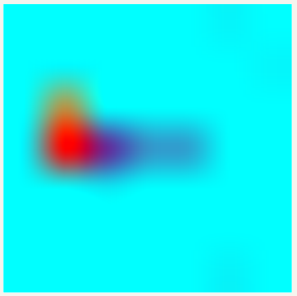
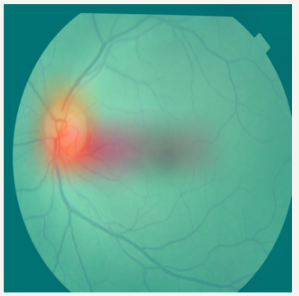
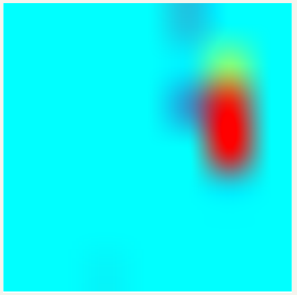
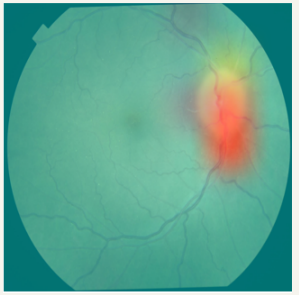
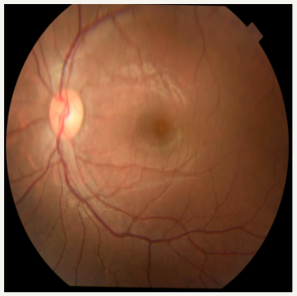
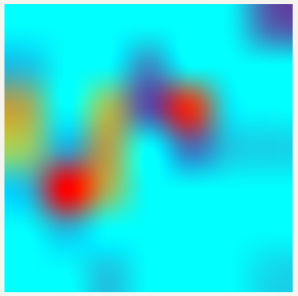
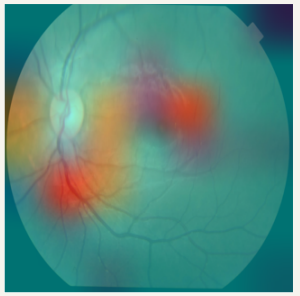
오류 분석의 우선순위

Myopia의 Recall은 두 전략 모두 0.7622였습니다. 질환 이미지를 Healthy로 분류하는 사례를 따로 모아 확인하는 것을 후속 과제로 정했습니다.

단일 저장 결과의 관측 차이이며 통계적 유의성이나 외부 일반화 성능을 뜻하지 않습니다. 표·그래프는 평가 CSV에서 재구성했습니다.

Grad-CAM으로 모델 주목 영역 점검

정답률과 별개로 모델이 이미지의 어느 영역을 강조하는지 비교했습니다.

	Original	Heatmap	Overlay
Glaucoma 시신경 유두 주변으로 집중하는 예시를 관찰			
Disc Edema 일부 영역에 집중하지만 샘플별 분산도 확인			
Myopia 주목 영역이 넓게 분산된 예시를 확인			

선정 기준: 기존 포트폴리오의 대표 3개 클래스에서 각각 저장 예시 00을 동일하게 사용했습니다.
정성적 관찰이며 병변 위치 정답·전문의 평가로 검증한 결과는 아닙니다.

이미지 업로드부터 결과 전달까지

Gradio 화면과 Hugging Face Spaces 데모로 추론 결과를 확인할 수 있는 흐름을 구현했습니다.

01

이미지 입력

안저 이미지 업로드
RGB 변환·정규화

02

모델 추론

ConvNeXtV2 Tiny
10개 클래스 예측

03

결과 해석

Top-3 예측·확률
Grad-CAM 생성

04

화면 표시

입력·heatmap·overlay
Gradio로 전달

demo/app.py | 추론 흐름 발췌

```
original_image = image.convert("RGB")
input_tensor = TRANSFORM(original_image)
input_tensor = input_tensor.unsqueeze(0).to(DEVICE)
outputs = MODEL(input_tensor)
probs = F.softmax(outputs, dim=1)
top3_idx = probs.cpu().numpy()[0].argsort()[-3:][::-1]
```

구현 코드의 흐름을 줄여 표기 / 학습·추론 재실행은 하지 않음

이미지 분류 문제로 이어갈 경험

입력 품질 점검, 클래스 불균형 대응, 클래스별 오류 분석, 시각적 결과 설명을 연결했습니다. 다른 이미지 분류 문제에도 데이터와 평가 조건을 먼저 정의하는 방식을 적용하고자 합니다.

다음 검증

원본 단위 분할과 외부 데이터 평가
학습·추론 전처리 조건 및 공개 실행 코드 정비
입력 품질 검사와 예측 확률 표시 방식 개선

연구용 프로토타입입니다. best.pt는 GitHub에 없으며, 공개 저장소만으로 즉시 학습·추론이 재현되는 상태는 아닙니다.